Enterprise Security Homelab Project

by Anastasia Alnykin

Overview

For the first step in building our homelab, we will establish the core environment. This includes configuring a Windows workstation, a Linux workstation, a corporate server, an Active Directory server, and an email server. These systems will form the foundational infrastructure used throughout the lab for administration, authentication, and service hosting.

Environments
Windows 11 Enterprise
Windows Server 2025
Ubuntu 22.04 Desktop
Oracle Linux 9

Tools
VMware Workstation Player/Pro
Samba Winbind
Docker
MailHog

Setup Details

Domain Controller - [8d6-labs-dc]
Acts as the core infrastructure server, running Active Directory Domain Services, DNS, DHCP, File and Storage Services, and IIS to provide centralized authentication, networking, shared storage, and internal web hosting.

Corporate Server - [8d6-labs-corp-svrt]
This is our jumpbox server. From here, we provision and access services such as FTP, DNS, and email. By limiting administrative access through a dedicated jumpbox, we reduce the network's attack surface.

Security Server - [8d6-labs-sec-box]
Serves as a domain-connected security management host. Centralizes security tooling, logs, alerts, and vulnerability data for the environment. Separating it from the main server improves performance, protects data, and reflects realistic deployment practices.

Security Workstation [8d6-labs-sec-work]
This is our Security Onion system. It acts as the analyst-facing monitoring box for reviewing network activity and conducting investigation.

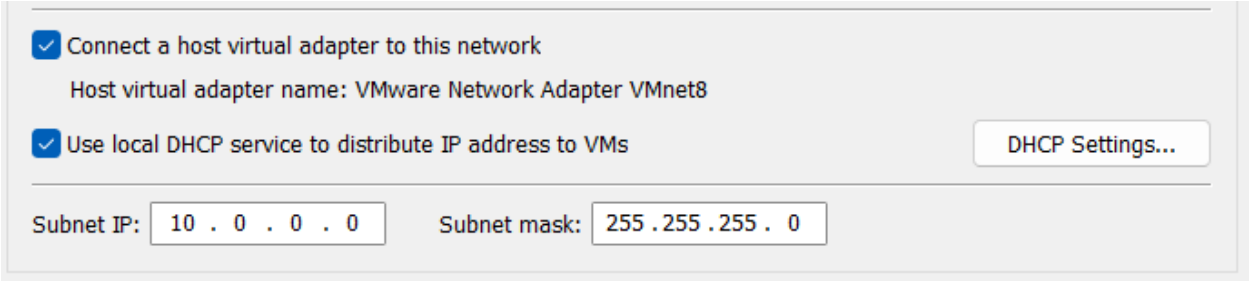
Windows Workstation - [8d6-labs-win-client] - Emulates a Windows host.
Linux Workstation - [8d6-labs-linux-client] - Emulates a Linux host.

Step 1: Configuring the Network

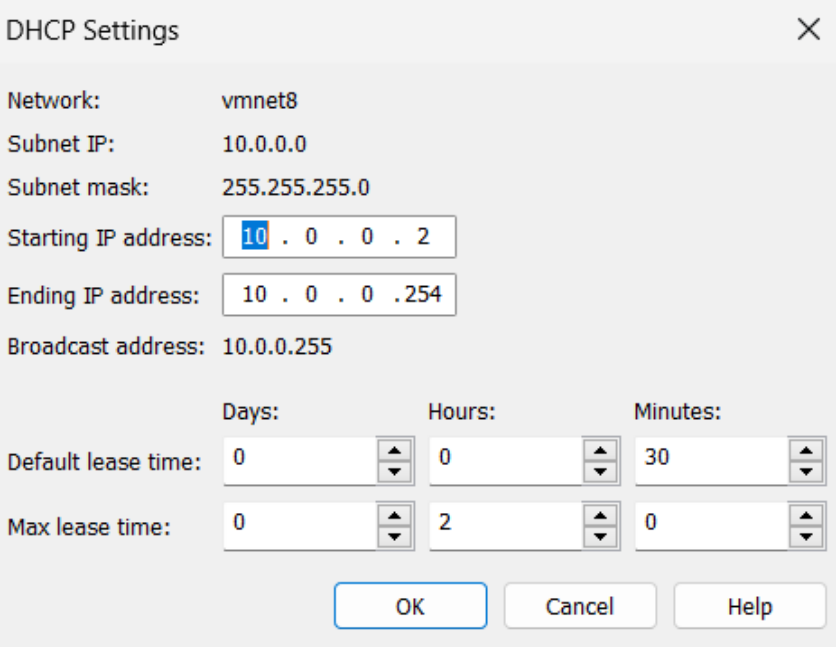
VMWare's Virtual Network Editor allows us to define custom virtual networks, which can be configured for internal-only traffic, Network Address Translation (NAT), or completely custom routing. The **Custom VMnet (NAT-enabled) network** configuration allows our VMs to access the internet while maintaining a logical separation from our physical network.

Network Name: VMnet8 (NatNetwork)
IP Address Range: 10.0.0.0/24
    Usable Range: 10.0.0.1 - 10.0.0.254
    DHCP Dynamic Scope: 10.0.0.100 - 10.0.0.200

To start, we will use the default VMNet8 NAT network to allow us to connect to our VMs as well as the Internet. We will need to change the IPs to conform to our IP address space.



Subnet IP to 10.0.0.0



DHCP Starting IP to 10.0.0.2 and Ending IP to 10.0.0.254

Step 2: Directory Services and Workstations

Building a Directory Services Server

[8d6-labs-dc]

- Download Windows Server 2025 onto VM.
- Configure Internet adapter settings with a static IP address.
- Install Active Directory Domain Services, DHCP Server, DNS Server, File and Storage Services, and Web Server (IIS).
- Configure the domain controller: corp.8d6-labs-dc.com.
- Configure DNS to give our domain controller Internet access.
- Configure DHCP.

Setting up the Windows Client

[8d6-labs-win-client]

- Download Windows 11 Enterprise.
- Change network connection to host-only & run oobe\bypassnro to to bypass Microsoft account sign in. Then return to NatNetwork.
- Configure static IP & DNS.
- Update computer name and connect to domain.
- Login as user *johnd*.

Setting up the Ubuntu Linux Client

[8d6-labs-linux-client]

- Download Ubuntu.
- Setup user *janed*.
- Configure static IP & DNS.
- Connect Linux Client to Active Directory.

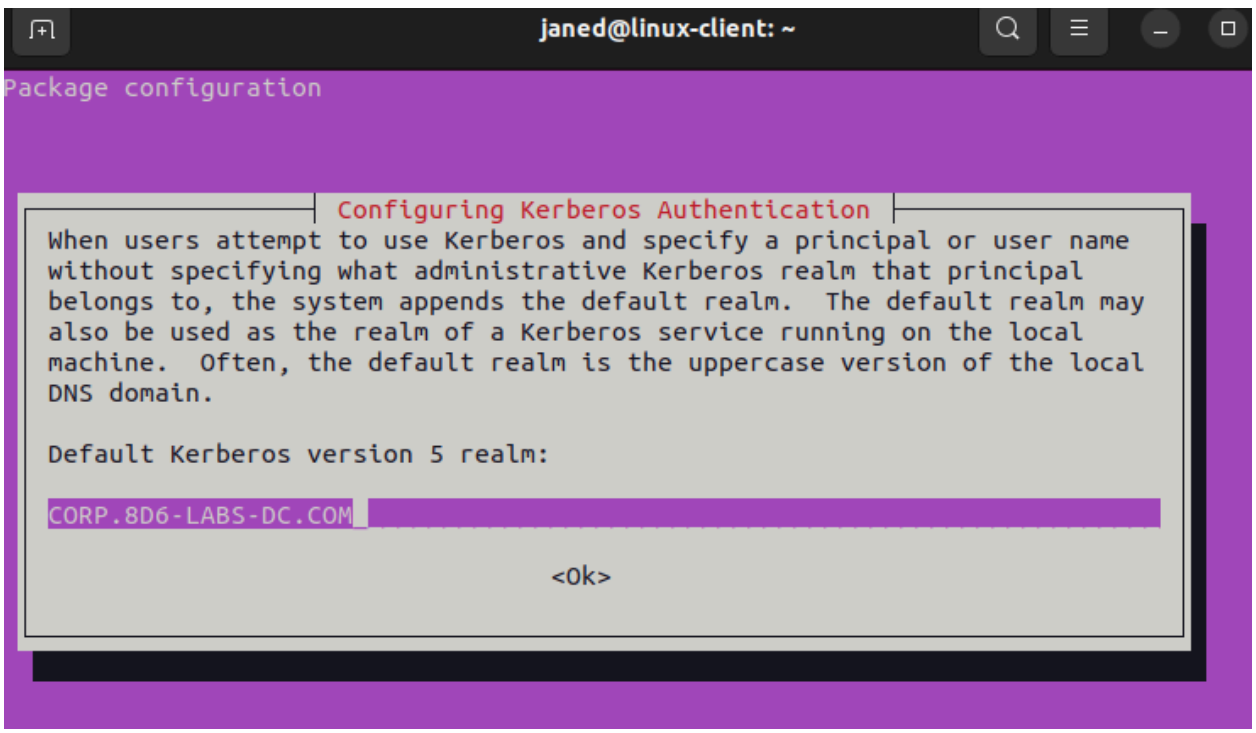
Step 2.5: Connecting the Linux Client to Active Directory

Samba Winbind is part of the Samba suite which we will use to enable our Linux system to authenticate against Windows Active Directory (AD) and integrate with our Windows network environment. It's also useful for legacy systems needing tight Windows protocol compatibility.

In a Terminal session, we first update the system with the `sudo apt upgrade` command. Then, install the required dependencies and packages for Samba Winbind:

```
sudo apt -y install winbind libpam-winbind libnss-winbind krb5-config samba-dsdb-modules samba-vfs-modules
```

The 'Configuring Kerberos Authentication' dialogue box will appear, here we enter our domain controller address in all caps.



Move the *smb.conf* file to *smb.conf.org* to replace the contents of *smb.conf* with our configurations instead.

```
sudo mv /etc/samba/smb.conf /etc/samba/smb.conf.org
```

Create and open the smb.conf file with Nano.

```
sudo nano /etc/samba/smb.conf
```

We replace realm and workgroup with the following (note spacing):

[global]
kerberos method = secrets and keytab
realm = CORP.8D6-LABS-DC.COM
workgroup = CORP
security = ads
template shell = /bin/bash
winbind enum groups = Yes
winbind enum users = Yes
winbind separator = +
*idmap config * : rangesize = 1000000*
*idmap config * : range = 1000000-19999999*
*idmap config * : backend = autorid*

Confirm *passwd* and *group* blocks have *winbind* set as a value.

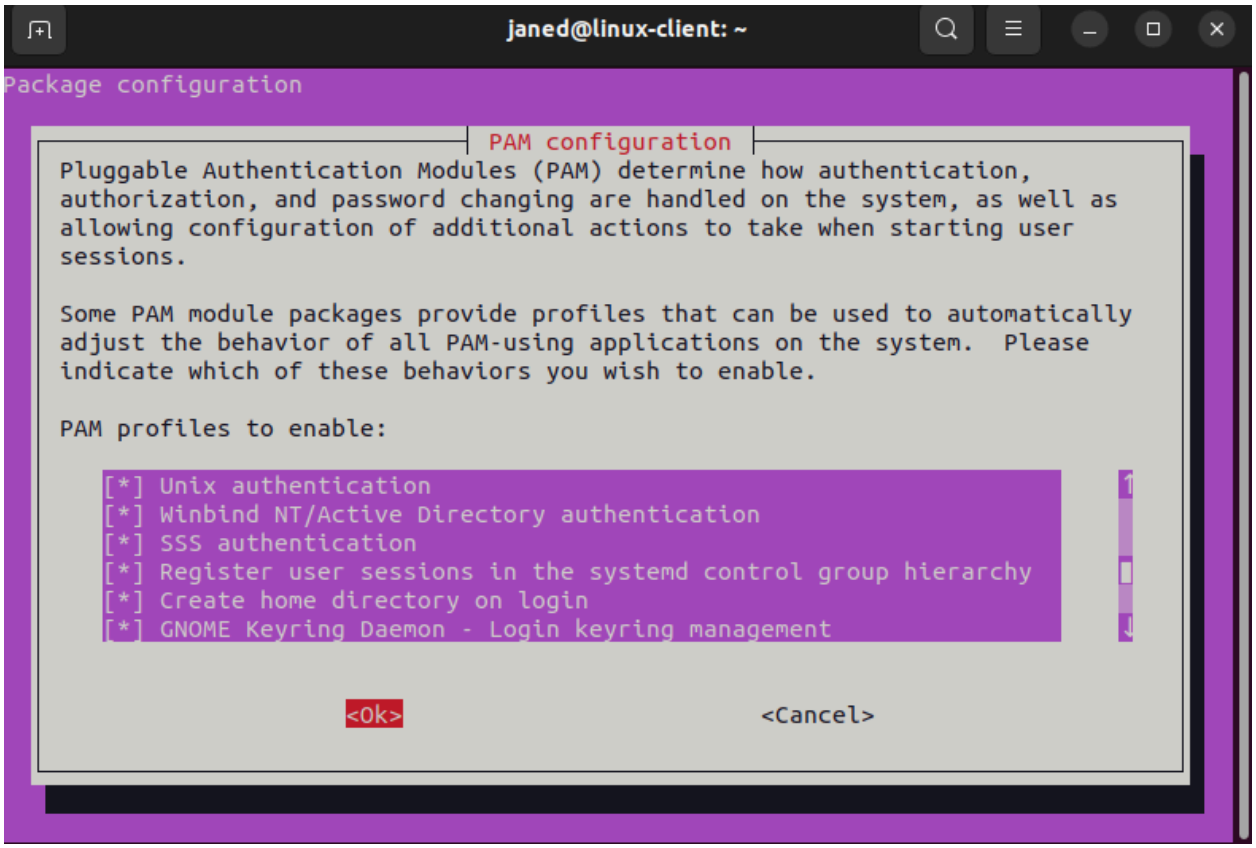
```
sudo nano /etc/nsswitch.conf
```

```
passwd:         files systemd sss winbind
group:          files systemd sss winbind
shadow:         files sss
gshadow:        files
```

On Ubuntu, users with interactive logons require a home directory. For domain users, this must be set before they can successfully log on and begin work.

```
sudo pam-auth-update
```

Scroll down to "Create home directory on login". Use the space bar to select with the *, hit the tab key to “OK” and hit Enter.



Change DNS settings to refer to AD:

```
sudo nano /etc/resolv.conf
```

```
nameserver 10.0.0.5
nameserver 127.0.0.53
options edns0 trust-ad
search .
```

Join the domain with Administrator:

```
sudo net ads join -U Administrator
```

```
janed@linux-client:~$       sudo net ads join -U Administrator
Password for [CORP\Administrator]:
Using short domain name -- CORP
Joined 'LINUX-CLIENT' to dns domain 'corp.8d6-labs-dc.com'
No DNS domain configured for linux-client. Unable to perform DNS Update.
DNS update failed: NT_STATUS_INVALID_PARAMETER
```

Restart winbind:

```
systemctl restart winbind
```

Get Active Directory services information listing:

```
net ads info
```

```
janed@linux-client:~$       net ads info
Failed to open /var/lib/samba/private/secrets.tdb
LDAP server: 10.0.0.5
LDAP server name: 8d6-labs-dc.corp.8d6-labs-dc.com
Realm: CORP.8D6-LABS-DC.COM
Bind Path: dc=CORP,dc=8D6-LABS-DC,dc=COM
LDAP port: 389
Server time: Tue, 03 Mar 2026 15:33:32 EST
KDC server: 10.0.0.5
Server time offset: 37
Last machine account password change: Wed, 31 Dec 1969 19:00:00 EST
```

List all available users:

```
wbinfo -u
```

```
janed@linux-client:~$       wbinfo -u
CORP+administrator
CORP+guest
CORP+krbtgt
CORP+johnd
CORP+janed
```

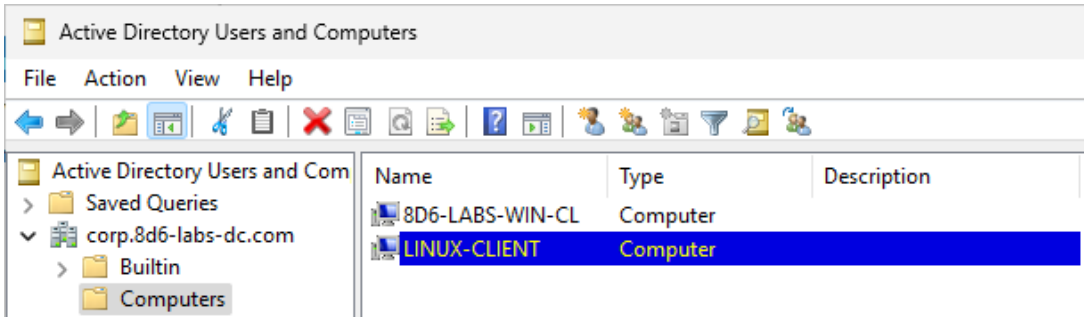
Login as janed:

```
sudo login
```

```
janed@linux-client:~$ sudo login
linux-client login: CORP+janed
Password:
```

```
Creating directory '/home/CORP/janed'.
CORP+janed@linux-client:~$ id
uid=2001107(CORP+janed) gid=2000513(CORP+domain users) groups=2000513(CORP+domain user
s),2001107(CORP+janed)
```

If we check back on our Active Directory server, we can see that the Linux client has been successfully connected. At this point, we save another snapshot of the system.



Step 3: Corporate Server

We will provision a dedicated server to serve as a **jump box**. It will be the sole access point for provisioning and managing services such as FTP, DNS, and email, reducing our network's attack surface.

Newer cybersecurity technology such as SSH tunneling and privileged access management solutions are more commonly used today, but a jumpbox still offers valuable capabilities to centralize and control administrative access.

Since we already have a fully provisioned linux client, we will clone it and use it as a baseline for our new server.

Setting up the Corporate Server

[8d6-labs-corp-svr]

- Clone [8d6-labs-linux-client].
- Set a unique static IP and change the hostname.
- Create a server admin account and provide sudo access.
- Login to admin, join the server under a new Active Directory domain account.
- Create a new home directory folder (complete AD enrollment.)
- Install Docker engine.

Step 4. Email Server

With our [8d6-labs-corp-svr] VM set up and Doctor installed, we can launch our first service - an email server. We will be using MailHog for sending and routing emails to our workstations.

MailHog is a simple yet powerful email testing utility. It functions as a mock SMTP server, capturing all outbound application emails instead of routing them to live mailboxes. We can then easily review these captured emails through a web-based interface or an API.

MailHog Specifics

SMTP Server - Listens on port **1025** by default

- Applications can send mail via **localhost:1025**

Web UI - Accessible on port **8025**

- Allows viewing of messages, headers, and attachments

API - Used for scripting and test automation.

The UI and API can be accessed using *curl* or a web browser.

Setting up the Email Server

MailHog is conveniently available as an official, ready-to-use Docker image on Docker Hub, allowing for easy pulling and running via a Docker container.

Create a new directory for MailHog in the /home folder:

```
mkdir mailhog
```

Create a new Docker Compose file:

```
sudo nano docker-compose.yml
```

Paste in the following:

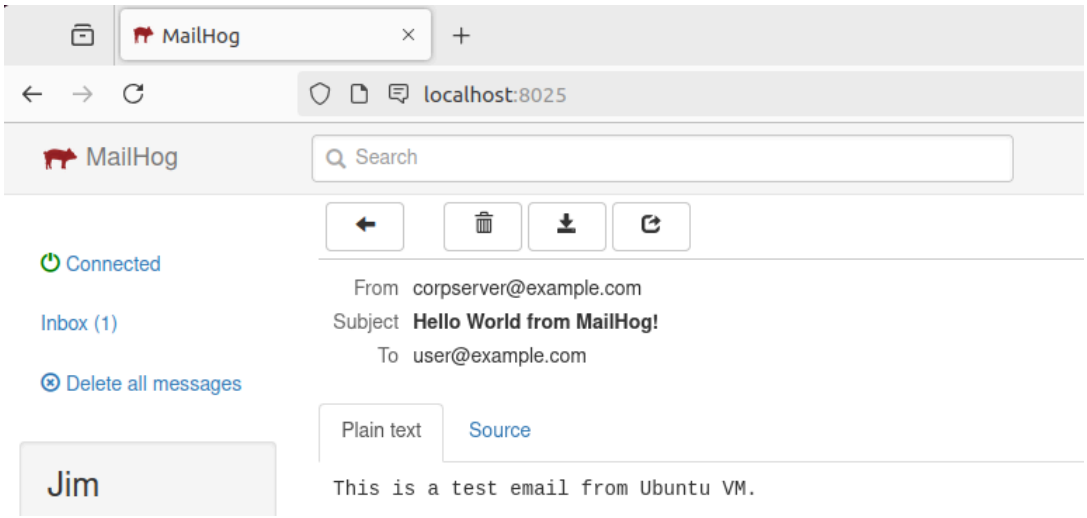
```
version: "3"
services:
  mailhog:
    image: mailhog/mailhog
    container_name: mailhog
    ports:
      - "1025:1025"
      - "8025:8025"
```

Run the container detached mode:

```
sudo docker compose up -d
```

```
janed@corp-svr:/home/mailhog$ sudo docker compose up -d
[sudo] password for janed:
WARN[0000] /home/mailhog/docker-compose.yml: the attribute `version` is obsolete
, it will be ignored, please remove it to avoid potential confusion
[+] up 10/10
 ✔ Image mailhog/mailhog    Pulled                                          11.3s
 ✔ Network mailhog_default  Created                                          0.0s
 ✔ Container mailhog        Started                                          0.7s
```

If we navigate to http://localhost:8025 we can now see the MailHog UI.



Successfully generated test email.

MailHog also allows us to view the source and headers.

Creating An Email Poller Script

With the basic MailHog SMTP server set up on **[8d6-labs-corp-svr]**, my next step is to simulate an **email polling service** on **[8d6-labs-linux-client]**. This will allow us to receive new email notifications on our Linux host.

First, we need to download some necessary packages. *curl* to fetch our email server URL. *jq* to filter JSON strings.

Then create a new Bash script file *email_poller.sh* with Nano with the following script:

```
#!/bin/bash

MAILHOG_IP="10.0.0.8"
TO_EMAIL="janed"
POLL_INTERVAL=30  # seconds

echo "📡 Janed's Mail Watcher started... polling every $POLL_INTERVAL seconds"
echo "🔎 Watching for new mail sent to: $TO_EMAIL@"

# Keep track of seen message IDs
SEEN_IDS_FILE="/tmp/mailhog_seen_ids_janed.txt"
touch "$SEEN_IDS_FILE"

while true; do
  # Fetch current message list
  curl -s http://$MAILHOG_IP:8025/api/v2/messages | jq -c '.items[]' | while read -r msg; do
    TO=$(echo "$msg" | jq -r '.To[].Mailbox')
    ID=$(echo "$msg" | jq -r '.ID')

    if [[ "$TO" == "$TO_EMAIL" && ! $(grep -Fx "$ID" "$SEEN_IDS_FILE") ]]; then
      SUBJECT=$(echo "$msg" | jq -r '.Content.Headers.Subject[0]')
      BODY=$(echo "$msg" | jq -r '.Content.Body')

      echo -e "\n📬 New Email Received!"
      echo "Subject: $SUBJECT"
      echo "From: $(echo "$msg" | jq -r '.Content.Headers.From[0]')"
      echo "Date: $(echo "$msg" | jq -r '.Created')"
      echo -e "Message:\n$BODY"
      echo "-----------------------------------"

      echo "$ID" >> "$SEEN_IDS_FILE"
    fi
  done

  sleep "$POLL_INTERVAL"
done
```

Make *email_poller.sh* executable using `chmod`, and run it in the background. Returning to **[8d6-labs-corp-svr]**, we generate a test email which successfully shows up on our Linux host, indicating the polling script is working.

```
janed@linux-client:/home$
📬 New Email Received!
Subject: Hello World from MailHog!
From: corpserver@example.com
Date: 2026-03-05T16:30:26.609741923Z
Message:
This is a test email from Ubuntu VM.
-----------------------------------
```

Step 5: Security Onion

Security Onion is an open-source Linux distribution specifically designed for threat hunting, enterprise security monitoring, and log management. It comes preconfigured with a suite of different security tools, including Zeek, Suricata, Elastic Stack, CyberChef, and a custom web interface.

Setting up Security Onion

- Run the installer on a new VM
- Set IP and Default Gateway
- Set DNS and connect to our corp domain.
- Restart the machine, we should now be able to login into Security Onion.

Step 6: SIEM

Wazuh is an open-source platform that provides **extended detection response (XDR)** and **System Information and Event Management (SIEM)**. It uses an agent-based ecosystem where agents are deployed on workstations, servers, containers, and virtual machines and send data to Wazuh's server for processing, aggregation, and visualization.

Agent configuration in Wazuh can be managed in two main ways:

1. **Centralized:** Using the `agent.conf` file for configuration that applies to multiple agents.
2. **Local:** Configuring individual agents directly via the `ossec.conf` file on each agent.

Before proceeding, we will first add **8d6-labs-sec-box** to our domain:

```
janed@sec-box:~/Desktop$ sudo net ads join -U Administrator
Password for [CORP\Administrator]:
Using short domain name -- CORP
Joined 'SEC-BOX' to dns domain 'corp.8d6-labs-dc.com'
```

```
janed@sec-box:~/Desktop$ wbinfo -u
CORP+administrator
CORP+guest
CORP+krbtgt
CORP+johnd
CORP+janed
CORP+secuser
```

```
Creating directory '/home/CORP/secuser'.
CORP+secuser@sec-box:~$
```

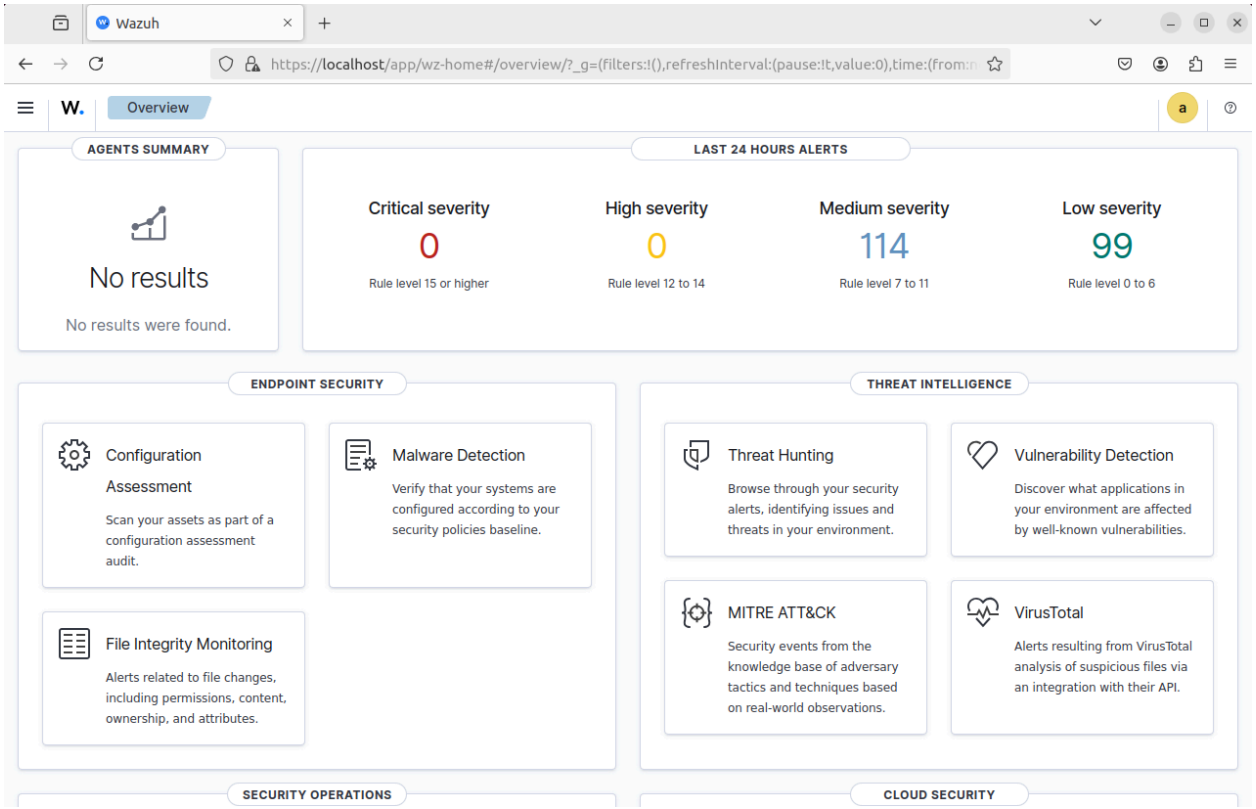
```
CORP+secuser@sec-box:~$ id
uid=2001111(CORP+secuser) gid=2000513(CORP+domain users) groups=2000513(CORP+dom
ain users),1000000(BUILTIN+administrators),1000001(BUILTIN+users),2000512(CORP+d
omain admins),2000572(CORP+denied rodc password replication group),2001111(CORP+
secuser)
```

Setup Wazuh Indexer + Server

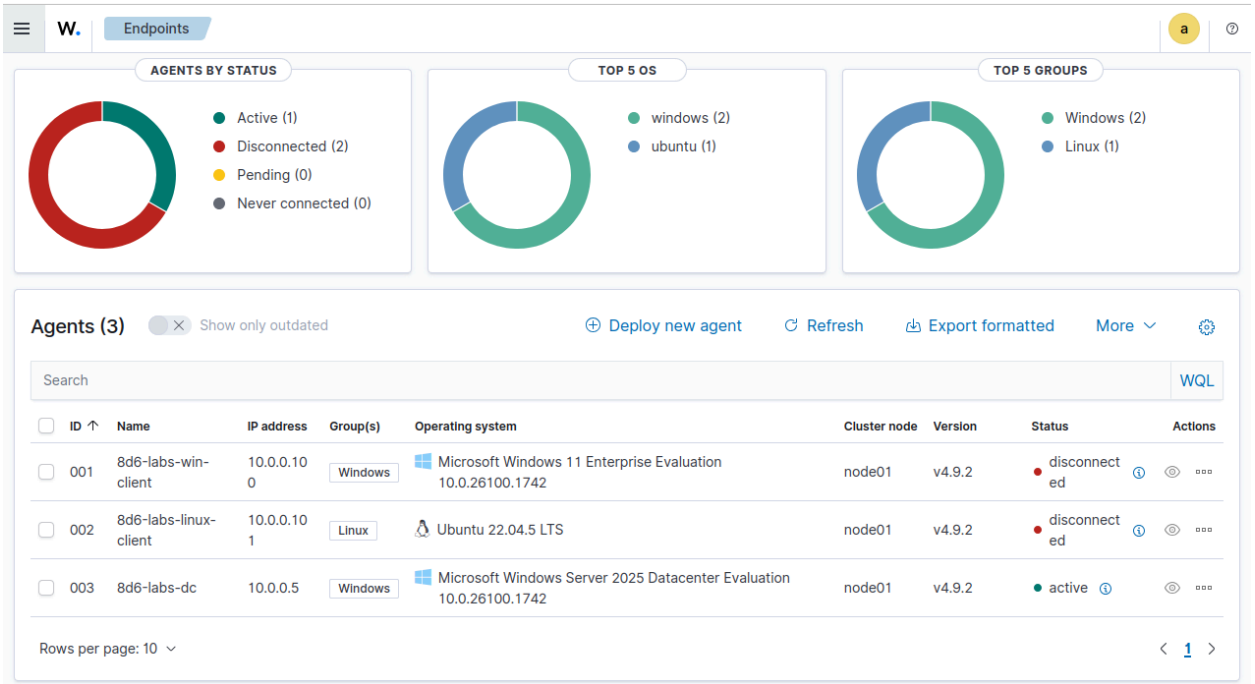
- We install the Wazuh indexer and server on the sec-box system by switching to sec-user, installing curl if needed, and running the Wazuh installation script.
- After installation finishes, note the generated admin password and credentials.
- Open https://localhost in a browser to access the Wazuh dashboard, accept the browser security warning, and log in.

Deploy Wazuh Agents

- From the Wazuh dashboard, deploy new agents to the Windows and Linux endpoints by selecting the correct installer type, setting the server address to 10.0.0.10, assigning each system its own agent name, and keeping the group as default.
- On each endpoint, we then run the generated install command with admin privileges, then start or enable the Wazuh agent service so the systems register with the Wazuh server.
- This is repeated for project-x-win-client, project-x-dc, and project-x-linux-client.



Wazuh Dashboard after successful installation.



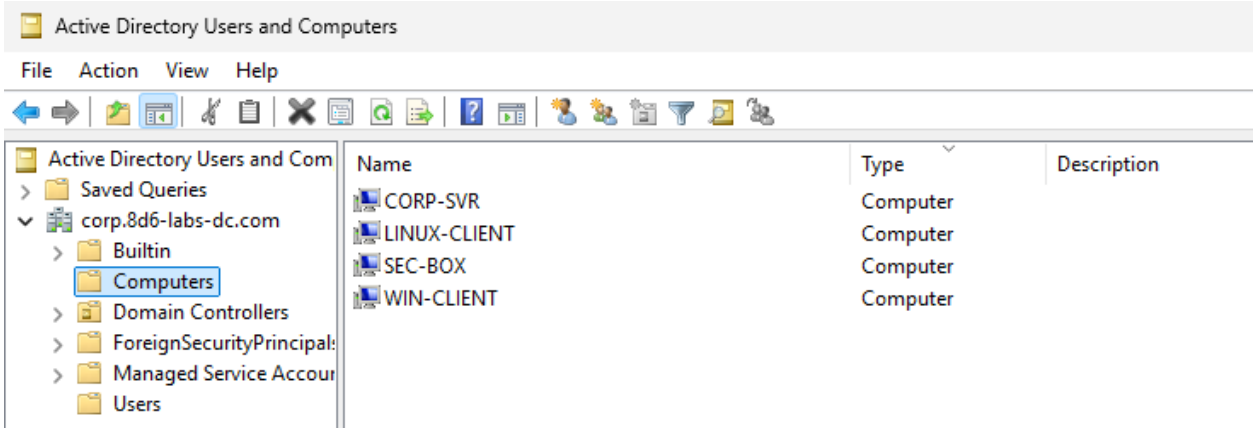
Overview of deployed agents.

Conclusion

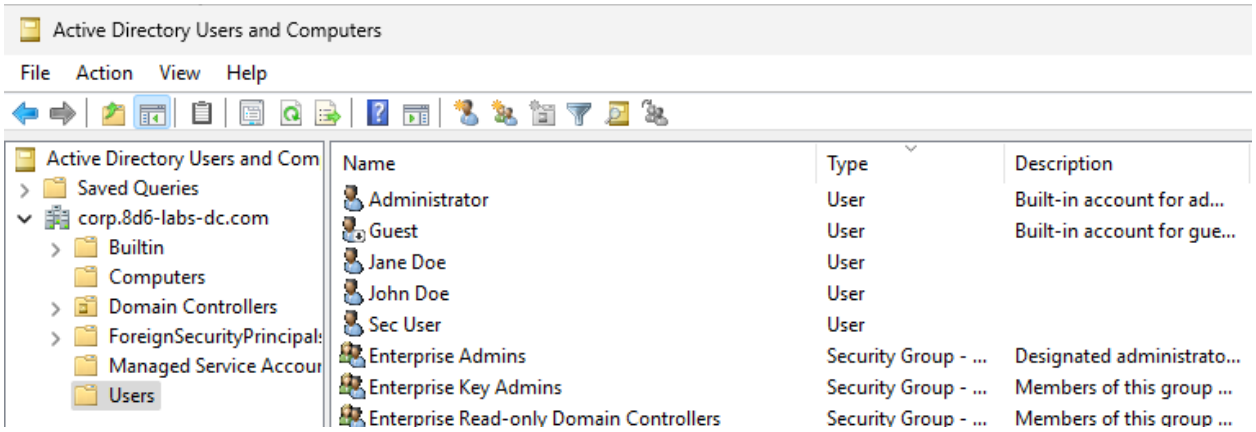
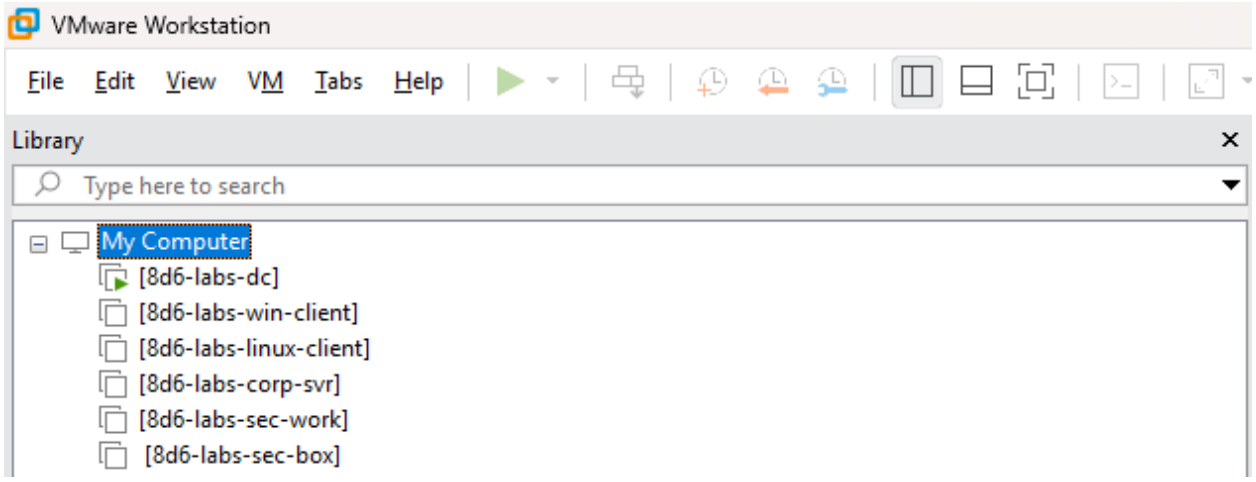
Building and troubleshooting this homelab gave me a better understanding of how enterprise systems are deployed, connected, and maintained, and showed me how much day-to-day security depends on having a solid administrative and infrastructure foundation. It also reinforced the importance of patience, problem-solving, and learning systems by working through them directly. This environment provides a flexible platform for continued learning and testing, and I look forward to expanding it with additional monitoring, hardening, and practical scenarios. Special thanks to Grant Collins for his Enterprise 101 course, which this project is based on.

End States

Active Directory with Connected Computers



Active Directory with Connected Users



Virtual Machines